UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Ingeniería de Sistemas · Bases de Datos Avanzadas

Docente: René Alejandro Lobo Quintero

Stanke

Plataforma de gestión y distribución digital de videojuegos

Proyecto Final — Entrega 2

Modelo de datos, implementación y lógica en la base de datos

Integrantes

Nombre completo	Código	Correo institucional
Juan David Escallon Guzman	20231020002	jdescallong@udistrital.edu.co
Juan Pablo Sarmiento Ramírez	20221020138	jpsarmientor@udistrital.edu.co

Asignatura: Bases de Datos Avanzadas

Grupo: _____

Fecha: _____

Contenido de la entrega

Este documento acompaña la carpeta `sql/` del archivo comprimido. Los archivos `01_create.sql`, `02_datos.sql` y `03_procedimientos_triggers.sql` se ejecutan completos, en ese orden y sin errores sobre una base de datos vacía; `04_pruebas.sql` contiene las pruebas de los componentes 2.3 y 2.4 y se ejecuta sentencia por sentencia. Todas las salidas que se muestran son salidas reales de psql sobre PostgreSQL 18 (los mensajes del servidor están en inglés, como en el ejemplo del enunciado).

Archivo	Contenido
`sql/01_create.sql`	14 tablas con llaves primarias y foráneas, CHECK, UNIQUE, DEFAULT e índices. Reglas RN-01 a RN-11.
`sql/02_datos.sql`	194 filas de prueba coherentes con las reglas; 30 compras en la tabla de mayor movimiento.
`sql/03_procedimientos_triggers.sql`	5 casos de uso (RF-03, RF-06, RF-14, RF-15, RF-17) y 5 triggers (RN-12 a RN-16).
`sql/04_pruebas.sql`	11 pruebas de restricciones, pruebas de éxito y error de cada caso de uso y de cada trigger, y prueba de atomicidad.

2.1 Modelo de datos

La Figura 1 muestra las 14 tablas de `01_create.sql` con sus columnas, tipos de dato, llaves primarias (PK), foráneas (FK), columnas únicas (UK) y la cardinalidad de cada relación en notación pata de gallo. El diagrama se genera a partir del mismo modelo del script, de modo que los nombres y tipos coinciden exactamente. La figura está en la página horizontal al final de esta sección.

Decisiones de diseño

Tablas intermedias. `videojuego_genero` y `promocion_videojuego` resuelven las relaciones muchos a muchos entre videojuegos y géneros, y entre promociones y videojuegos. Su llave primaria es la pareja de llaves foráneas, de modo que un juego no puede repetir un género ni aparecer dos veces en la misma promoción. `biblioteca` y `lista_deseos` también relacionan usuarios con videojuegos, pero significan cosas distintas (propiedad frente a interés), por eso son tablas separadas; ambas usan la pareja (usuario_id, videojuego_id) como llave primaria, lo que impide tener dos veces el mismo juego.

Compra frente a biblioteca. `compra` usa un identificador propio porque un jugador puede comprar, reembolsar y volver a comprar el mismo juego: es un registro histórico de dinero. `biblioteca` refleja solo lo que el jugador posee hoy; la llena el trigger de RN-14 y la vacía el reembolso (RF-17).

Historial y auditoría. `historial_precio` guarda cada cambio de precio con el precio anterior, el nuevo y la fecha (RN-16, RNF-09). Ninguna aplicación escribe en ella: la llena el trigger `trg_rn16_historial_precio`. Su llave foránea es ON DELETE RESTRICT para no perder la auditoría.

Borrado. Las tablas que dependen por completo de su padre (géneros e imágenes de un juego, biblioteca, lista de deseos, amistades, juegos de una promoción) usan ON DELETE CASCADE. Las que son registros de negocio (compras, reseñas, juegos de un desarrollador, historial) usan ON DELETE RESTRICT: no se puede borrar un usuario o un juego que ya tiene ventas.

Cambios respecto a la Entrega 1

- **Nombres de columnas.** La llave primaria de cada tabla se llama `id` y las llaves foráneas `<tabla>_id` (por ejemplo `usuario_id` en lugar de `id_usuario`). En `promocion`, `porcentaje_descuento` pasó a llamarse `descuento`.
- **Amistad.** Se agregó un identificador propio `id` y un índice único sobre la pareja de usuarios sin importar el orden (`LEAST/GREATEST`), para que no existan a la vez la solicitud A→B y la B→A.
- **Restricciones adicionales** que no eran reglas numeradas: un desarrollador no repite el título de sus juegos (`uq_videojuego_titulo_desarrollador`), las imágenes de un juego no repiten su orden, y el país del desarrollador es obligatorio.
- **RF-06.** Además de lo descrito en la Entrega 1, la compra quita el juego de la lista de deseos del jugador.
- **RF-17.** El reembolso también borra la reseña del jugador sobre ese juego; de lo contrario quedaría una reseña de un juego que ya no está en su biblioteca, en contra de RN-15.
- **RF-15** se implementa como función `RETURNS TABLE` y no como procedimiento, porque es una consulta que devuelve filas (lo permite el enunciado).
- Los requerimientos y reglas (RF-01 a RF-17, RN-01 a RN-16) conservan los identificadores de la Entrega 1.

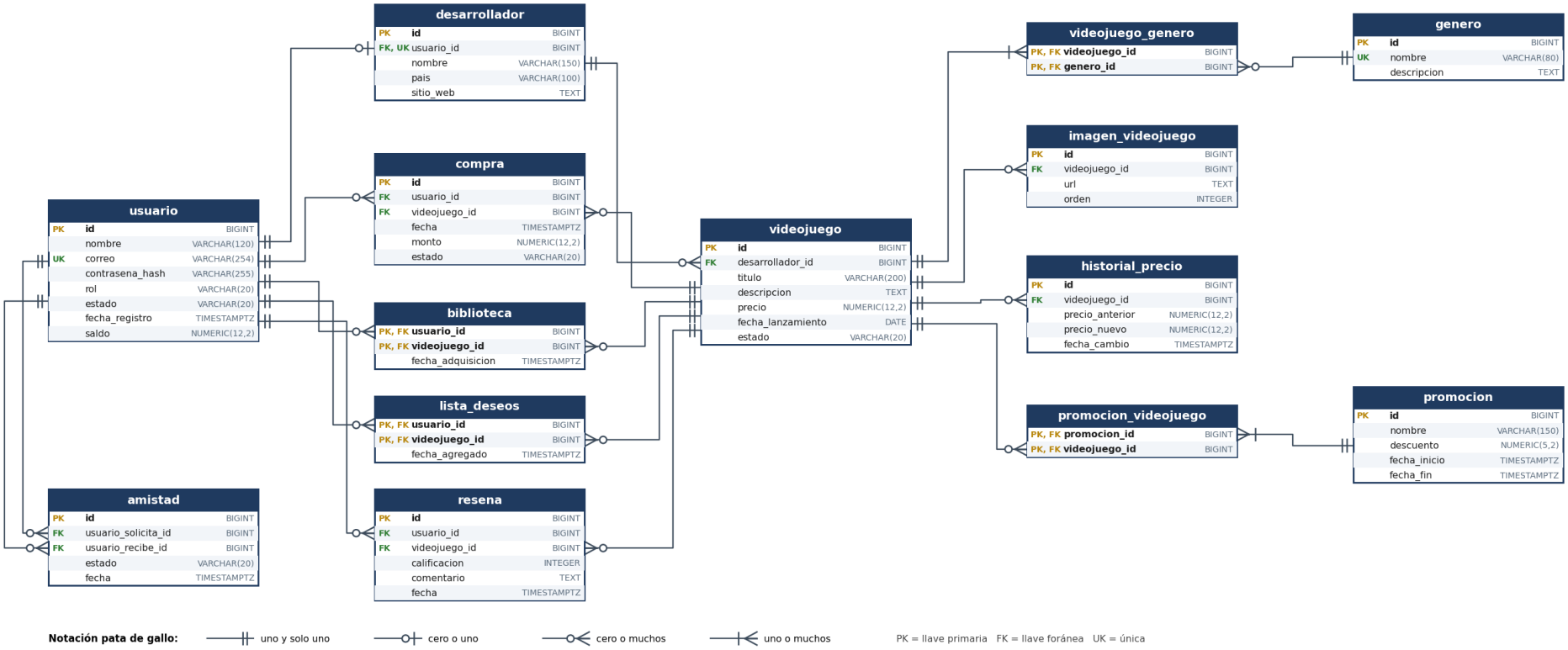


Figura 1. Modelo relacional de Stanke (coincide con 01_create.sql).

2.2 Script de creación

El archivo `01_create.sql` empieza con `DROP TABLE IF EXISTS … CASCADE` para cada tabla, por lo que puede ejecutarse varias veces. Todas las tablas tienen llave primaria; cada relación del diagrama es una llave foránea con ON DELETE RESTRICT o ON DELETE CASCADE explícito, y cada restricción tiene nombre propio para que los mensajes de error indiquen qué regla se violó. Las reglas declarativas de la Entrega 1 se marcan en el script con el comentario `-- RN-xx`.

Reglas de negocio implementadas como restricciones

Regla	Tabla	Restricción que la implementa
RN-01	usuario, desarrollador, videojuego, genero, compra, resena, promocion	`id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY`
RN-02	usuario	`uq_usuario_correo UNIQUE (correo)`
RN-03	usuario	`ck_usuario_rol CHECK (rol IN ('jugador', 'desarrollador', 'administrador'))`
RN-04	usuario	`ck_usuario_saldo CHECK (saldo >= 0)`
RN-05	videojuego	`ck_videojuego_precio CHECK (precio >= 0)`
RN-06	videojuego	`desarrollador_id BIGINT NOT NULL` + `fk_videojuego_desarrollador REFERENCES desarrollador(id) ON DELETE RESTRICT`
RN-07	videojuego	`ck_videojuego_estado CHECK (estado IN ('pendiente', 'publicado', 'retirado'))` `estado … DEFAULT 'pendiente'`
RN-08	resena	`calificacion INTEGER` + `ck_resena_calificacion CHECK (calificacion BETWEEN 1 AND 5)`
RN-09	resena	`uq_resena_usuario_videojuego UNIQUE (usuario_id, videojuego_id)`
RN-10	promocion	`ck_promocion_descuento CHECK (descuento BETWEEN 1 AND 90)` `ck_promocion_fechas CHECK (fecha_fin > fecha_inicio)`
RN-11	amistad	`ck_amistad_distintos CHECK (usuario_solicita_id <> usuario_recibe_id)`

Las reglas RN-12 a RN-16 dependen de otras tablas o producen efectos en otras tablas, por lo que se implementan con triggers en `03_procedimientos_triggers.sql` (sección 2.4).

Script completo (01_create.sql)

```
-- ================================================================
-- Stanke - Plataforma de gestión y distribución digital de videojuegos
-- 01_create.sql  ·  Creación de tablas, llaves y restricciones
-- PostgreSQL 14 o superior (probado en PostgreSQL 18)
--
-- Las reglas de negocio declarativas (RN-01 a RN-11) se señalan con un
-- comentario -- RN-xx en la línea que las implementa.
-- Las reglas RN-12 a RN-16 necesitan triggers y están en
-- 03_procedimientos_triggers.sql.
-- ================================================================

DROP TABLE IF EXISTS historial_precio      CASCADE;
DROP TABLE IF EXISTS promocion_videojuego  CASCADE;
DROP TABLE IF EXISTS promocion             CASCADE;
DROP TABLE IF EXISTS amistad               CASCADE;
DROP TABLE IF EXISTS resena                CASCADE;
DROP TABLE IF EXISTS lista_deseos          CASCADE;
DROP TABLE IF EXISTS biblioteca            CASCADE;
```

```
DROP TABLE IF EXISTS compra               CASCADE;
DROP TABLE IF EXISTS imagen_videojuego    CASCADE;
DROP TABLE IF EXISTS videojuego_genero    CASCADE;
DROP TABLE IF EXISTS genero               CASCADE;
DROP TABLE IF EXISTS videojuego           CASCADE;
DROP TABLE IF EXISTS desarrollador        CASCADE;
DROP TABLE IF EXISTS usuario              CASCADE;

-- ---------------------------------------------------------------------
-- Usuarios de la plataforma (jugadores, desarrolladores y administradores)
-- ---------------------------------------------------------------------
CREATE TABLE usuario (
    id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,          -- RN-01
    nombre          VARCHAR(120) NOT NULL,
    correo          VARCHAR(254) NOT NULL,
    contrasena_hash VARCHAR(255) NOT NULL,
    rol             VARCHAR(20)  NOT NULL,
    estado          VARCHAR(20)  NOT NULL DEFAULT 'activo',
    fecha_registro  TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    saldo           NUMERIC(12,2) NOT NULL DEFAULT 0,
    CONSTRAINT uq_usuario_correo  UNIQUE (correo),                             -- RN-02
    CONSTRAINT ck_usuario_rol     CHECK (rol IN ('jugador', 'desarrollador', 'administrador')), -- RN-03
    CONSTRAINT ck_usuario_estado  CHECK (estado IN ('activo', 'suspendido')),
    CONSTRAINT ck_usuario_saldo   CHECK (saldo >= 0)                           -- RN-04
);

-- ---------------------------------------------------------------------
-- Perfil de desarrollador / publicador (1:1 con un usuario)
-- ---------------------------------------------------------------------
CREATE TABLE desarrollador (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,              -- RN-01
    usuario_id  BIGINT       NOT NULL,
    nombre      VARCHAR(150) NOT NULL,
    pais        VARCHAR(100) NOT NULL,
    sitio_web   TEXT,
    CONSTRAINT uq_desarrollador_usuario UNIQUE (usuario_id),
    CONSTRAINT fk_desarrollador_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE RESTRICT
);

-- ---------------------------------------------------------------------
-- Catálogo de videojuegos
-- ---------------------------------------------------------------------
CREATE TABLE videojuego (
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,        -- RN-01
    desarrollador_id  BIGINT        NOT NULL,                                 -- RN-06
    titulo            VARCHAR(200)  NOT NULL,
    descripcion       TEXT,
    precio            NUMERIC(12,2) NOT NULL DEFAULT 0,
    fecha_lanzamiento DATE,
    estado            VARCHAR(20)   NOT NULL DEFAULT 'pendiente',             -- RN-07 (inicia en pendiente)
    CONSTRAINT ck_videojuego_precio CHECK (precio >= 0),                      -- RN-05
    CONSTRAINT ck_videojuego_estado
        CHECK (estado IN ('pendiente', 'publicado', 'retirado')),             -- RN-07
    CONSTRAINT uq_videojuego_titulo_desarrollador UNIQUE (desarrollador_id, titulo),
    CONSTRAINT fk_videojuego_desarrollador                                    -- RN-06
        FOREIGN KEY (desarrollador_id) REFERENCES desarrollador(id) ON DELETE RESTRICT
);

CREATE TABLE genero (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,              -- RN-01
    nombre      VARCHAR(80) NOT NULL,
    descripcion TEXT,
    CONSTRAINT uq_genero_nombre UNIQUE (nombre)
);

-- Relación N:M entre videojuego y género
CREATE TABLE videojuego_genero (
    videojuego_id BIGINT NOT NULL,
    genero_id     BIGINT NOT NULL,
    CONSTRAINT pk_videojuego_genero PRIMARY KEY (videojuego_id, genero_id),
    CONSTRAINT fk_videojuego_genero_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE CASCADE,
    CONSTRAINT fk_videojuego_genero_genero
        FOREIGN KEY (genero_id) REFERENCES genero(id) ON DELETE RESTRICT
);
```

```
CREATE TABLE imagen_videojuego (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    videojuego_id BIGINT  NOT NULL,
    url           TEXT    NOT NULL,
    orden         INTEGER NOT NULL DEFAULT 1,
    CONSTRAINT ck_imagen_orden CHECK (orden > 0),
    CONSTRAINT uq_imagen_videojuego_orden UNIQUE (videojuego_id, orden),
    CONSTRAINT fk_imagen_videojuego_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE CASCADE
);

-- ---------------------------------------------------------------------
-- Compras (tabla de mayor movimiento)
-- ---------------------------------------------------------------------
CREATE TABLE compra (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,             -- RN-01
    usuario_id    BIGINT        NOT NULL,
    videojuego_id BIGINT        NOT NULL,
    fecha         TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
    monto         NUMERIC(12,2) NOT NULL,
    estado        VARCHAR(20)   NOT NULL DEFAULT 'completada',
    CONSTRAINT ck_compra_monto  CHECK (monto >= 0),
    CONSTRAINT ck_compra_estado CHECK (estado IN ('completada', 'reembolsada')),
    CONSTRAINT fk_compra_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE RESTRICT,
    CONSTRAINT fk_compra_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE RESTRICT
);

-- Biblioteca: juegos que posee cada jugador (N:M usuario-videojuego)
CREATE TABLE biblioteca (
    usuario_id        BIGINT      NOT NULL,
    videojuego_id     BIGINT      NOT NULL,
    fecha_adquisicion TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_biblioteca PRIMARY KEY (usuario_id, videojuego_id),
    CONSTRAINT fk_biblioteca_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE CASCADE,
    CONSTRAINT fk_biblioteca_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE CASCADE
);

-- Lista de deseos (N:M usuario-videojuego)
CREATE TABLE lista_deseos (
    usuario_id     BIGINT      NOT NULL,
    videojuego_id  BIGINT      NOT NULL,
    fecha_agregado TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_lista_deseos PRIMARY KEY (usuario_id, videojuego_id),
    CONSTRAINT fk_lista_deseos_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE CASCADE,
    CONSTRAINT fk_lista_deseos_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE CASCADE
);

CREATE TABLE resena (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,             -- RN-01
    usuario_id    BIGINT      NOT NULL,
    videojuego_id BIGINT      NOT NULL,
    calificacion  INTEGER     NOT NULL,
    comentario    TEXT,
    fecha         TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_resena_calificacion CHECK (calificacion BETWEEN 1 AND 5),    -- RN-08
    CONSTRAINT uq_resena_usuario_videojuego UNIQUE (usuario_id, videojuego_id), -- RN-09
    CONSTRAINT fk_resena_usuario
        FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE RESTRICT,
    CONSTRAINT fk_resena_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE RESTRICT
);

CREATE TABLE amistad (
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    usuario_solicita_id BIGINT      NOT NULL,
    usuario_recibe_id   BIGINT      NOT NULL,
    estado              VARCHAR(20) NOT NULL DEFAULT 'pendiente',
    fecha               TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_amistad_estado
```

```
        CHECK (estado IN ('pendiente', 'aceptada', 'rechazada')),
    CONSTRAINT ck_amistad_distintos
        CHECK (usuario_solicita_id <> usuario_recibe_id),                      -- RN-11
    CONSTRAINT fk_amistad_usuario_solicita
        FOREIGN KEY (usuario_solicita_id) REFERENCES usuario(id) ON DELETE CASCADE,
    CONSTRAINT fk_amistad_usuario_recibe
        FOREIGN KEY (usuario_recibe_id) REFERENCES usuario(id) ON DELETE CASCADE
);
-- Una sola relación de amistad por pareja, sin importar quién la envió
CREATE UNIQUE INDEX uq_amistad_pareja
    ON amistad (LEAST(usuario_solicita_id, usuario_recibe_id),
                GREATEST(usuario_solicita_id, usuario_recibe_id));


CREATE TABLE promocion (
    id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,              -- RN-01
    nombre       VARCHAR(150) NOT NULL,
    descuento    NUMERIC(5,2) NOT NULL,
    fecha_inicio TIMESTAMPTZ  NOT NULL,
    fecha_fin    TIMESTAMPTZ  NOT NULL,
    CONSTRAINT ck_promocion_descuento CHECK (descuento BETWEEN 1 AND 90),      -- RN-10
    CONSTRAINT ck_promocion_fechas    CHECK (fecha_fin > fecha_inicio)          -- RN-10
);

-- Relación N:M entre promoción y videojuego
CREATE TABLE promocion_videojuego (
    promocion_id  BIGINT NOT NULL,
    videojuego_id BIGINT NOT NULL,
    CONSTRAINT pk_promocion_videojuego PRIMARY KEY (promocion_id, videojuego_id),
    CONSTRAINT fk_promocion_videojuego_promocion
        FOREIGN KEY (promocion_id) REFERENCES promocion(id) ON DELETE CASCADE,
    CONSTRAINT fk_promocion_videojuego_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE CASCADE
);

-- Auditoría de cambios de precio (la llena el trigger de RN-16)
CREATE TABLE historial_precio (
    id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    videojuego_id   BIGINT        NOT NULL,
    precio_anterior NUMERIC(12,2) NOT NULL,
    precio_nuevo    NUMERIC(12,2) NOT NULL,
    fecha_cambio    TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_historial_precio_anterior CHECK (precio_anterior >= 0),
    CONSTRAINT ck_historial_precio_nuevo    CHECK (precio_nuevo >= 0),
    CONSTRAINT fk_historial_precio_videojuego
        FOREIGN KEY (videojuego_id) REFERENCES videojuego(id) ON DELETE RESTRICT
);

-- ---------------------------------------------------------------------
-- Índices de apoyo para búsquedas y reportes (RF-05, RF-15, RNF-05)
-- ---------------------------------------------------------------------
CREATE INDEX ix_videojuego_desarrollador ON videojuego (desarrollador_id);
CREATE INDEX ix_videojuego_titulo        ON videojuego (lower(titulo));
CREATE INDEX ix_videojuego_genero_genero ON videojuego_genero (genero_id);
CREATE INDEX ix_compra_usuario           ON compra (usuario_id);
CREATE INDEX ix_compra_videojuego_fecha  ON compra (videojuego_id, fecha);
CREATE INDEX ix_resena_videojuego        ON resena (videojuego_id);
CREATE INDEX ix_promocion_videojuego_vj  ON promocion_videojuego (videojuego_id);
CREATE INDEX ix_historial_precio_vj      ON historial_precio (videojuego_id);
```

2.3 Datos de prueba

`02_datos.sql` carga una tienda con 15 usuarios (1 administrador, 5 estudios y 9 jugadores, uno de ellos suspendido), 11 videojuegos de 5 desarrolladores (9 publicados, 1 pendiente y 1 retirado), 6 promociones y 30 compras entre diciembre de 2025 y noviembre de 2026. Los datos se construyeron para que sean coherentes con todas las reglas aunque los triggers todavía no existan cuando se cargan:

- cada compra completada tiene su fila en `biblioteca` y las dos compras reembolsadas no (RN-14, RF-17);
- el monto de cada compra es el precio vigente en su fecha, con el descuento de la promoción activa (por ejemplo, 39.99 × 0.80 = 31.99 en las Rebajas de Verano);

- las 20 reseñas son de juegos que el autor posee y tienen fecha posterior a la compra (RN-15);
- las listas de deseos solo tienen juegos que el jugador aún no posee;
- las contraseñas son hashes bcrypt reales (costo 12), nunca texto plano (RNF-01);
- hay casos útiles para las pruebas y las preguntas de negocio: un juego publicado sin ventas (Sombras de Cartagena, P7), juegos en la lista de deseos con promoción vigente (P6), un juego gratuito retirado y una cuenta suspendida.

Conteo de filas después de ejecutar `01` y `02` (salida real):

```
        tabla         | filas
----------------------+-------
 usuario              |    15
 desarrollador        |     5
 videojuego           |    11
 genero               |     8
 videojuego_genero    |    18
 imagen_videojuego    |    14
 compra               |    30
 biblioteca           |    28
 lista_deseos         |    11
 resena               |    20
 amistad              |     7
 promocion            |     6
 promocion_videojuego |    15
 historial_precio     |     6
(14 rows)
```

Pruebas de restricciones

Cada sentencia se ejecutó sobre la base recién cargada y PostgreSQL la rechazó con el mensaje que se muestra. Se incluyen más de las tres exigidas: CHECK, llave foránea, ON DELETE RESTRICT, UNIQUE y NOT NULL.

R1 · CHECK · RN-08: calificación fuera del rango 1–5

```
INSERT INTO resena (usuario_id, videojuego_id, calificacion, comentario)
VALUES (9, 2, 6, 'Lo mejor que he jugado en mi vida.');
ERROR:  new row for relation "resena" violates check constraint "ck_resena_calificacion"
DETAIL:  Failing row contains (21, 9, 2, 6, Lo mejor que he jugado en mi vida., 2026-11-03 19:45:36.714073-05).
```

R2 · CHECK · RN-04: el saldo no puede quedar negativo

```
UPDATE usuario SET saldo = saldo - 500 WHERE id = 10;
ERROR:  new row for relation "usuario" violates check constraint "ck_usuario_saldo"
DETAIL:  Failing row contains (10, Sofía Martínez Díaz, sofia.martinez@gmail.com,
$2b$12$WiFxHxk8blWWCvwrQZaYKeIZNtfdOYynrUlt9cQaof9UblJA5yAje, jugador, activo, 2025-06-22 18:25:00-05, -484.70).
```

R3 · CHECK · RN-05: precio negativo

```
INSERT INTO videojuego (desarrollador_id, titulo, precio)
VALUES (1, 'Pixel Dungeon Tactics II', -5.00);
ERROR:  new row for relation "videojuego" violates check constraint "ck_videojuego_precio"
DETAIL:  Failing row contains (12, 1, Pixel Dungeon Tactics II, null, -5.00, null, pendiente).
```

R4 · CHECK · RN-10: descuento mayor que 90 %

```
INSERT INTO promocion (nombre, descuento, fecha_inicio, fecha_fin)
VALUES ('Liquidación total', 95, '2026-12-01', '2026-12-05');
ERROR:  new row for relation "promocion" violates check constraint "ck_promocion_descuento"
DETAIL:  Failing row contains (7, Liquidación total, 95.00, 2026-12-01 00:00:00-05, 2026-12-05 00:00:00-05).
```

R5 · CHECK · RN-11: solicitud de amistad a sí mismo

```
INSERT INTO amistad (usuario_solicita_id, usuario_recibe_id) VALUES (9, 9);
ERROR:  new row for relation "amistad" violates check constraint "ck_amistad_distintos"
DETAIL:  Failing row contains (8, 9, 9, pendiente, 2026-11-03 19:45:36.723343-05).
```

R6 · CHECK · RN-03: rol no permitido

```
INSERT INTO usuario (nombre, correo, contrasena_hash, rol)
VALUES ('Valeria Montoya', 'valeria.montoya@gmail.com',
        '$2b$12$bQy79C9Xn/Exh16v2uXxluyqYOMV59gt8cueaLthjJ/IEByY47j16', 'moderador');
ERROR:  new row for relation "usuario" violates check constraint "ck_usuario_rol"
DETAIL:  Failing row contains (16, Valeria Montoya, valeria.montoya@gmail.com,
$2b$12$bQy79C9Xn/Exh16v2uXxluyqYOMV59gt8cueaLthjJ/IEByY47j16, moderador, activo, 2026-11-03 19:45:36.723711-05,
0.00).
```

R7 · Llave foránea · RN-06: desarrollador inexistente

```
INSERT INTO videojuego (desarrollador_id, titulo, precio)
VALUES (99, 'Reino Perdido', 19.99);
ERROR:  insert or update on table "videojuego" violates foreign key constraint "fk_videojuego_desarrollador"
DETAIL:  Key (desarrollador_id)=(99) is not present in table "desarrollador".
```

R8 · Llave foránea · ON DELETE RESTRICT: borrar un desarrollador con juegos

```
DELETE FROM desarrollador WHERE id = 1;
ERROR:  update or delete on table "desarrollador" violates RESTRICT setting of foreign key constraint
"fk_videojuego_desarrollador" on table "videojuego"
DETAIL:  Key (id)=(1) is referenced from table "videojuego".
```

R9 · UNIQUE · RN-02: correo repetido

```
INSERT INTO usuario (nombre, correo, contrasena_hash, rol)
VALUES ('Carlos Andrés Pérez', 'carlos.perez@gmail.com',
        '$2b$12$bQy79C9Xn/Exh16v2uXxluyqYOMV59gt8cueaLthjJ/IEByY47j16', 'jugador');
ERROR:  duplicate key value violates unique constraint "uq_usuario_correo"
DETAIL:  Key (correo)=(carlos.perez@gmail.com) already exists.
```

R10 · UNIQUE · RN-09: segunda reseña del mismo jugador al mismo juego

```
INSERT INTO resena (usuario_id, videojuego_id, calificacion, comentario)
VALUES (7, 1, 4, 'Lo volví a jugar y sigue siendo excelente.');
ERROR:  duplicate key value violates unique constraint "uq_resena_usuario_videojuego"
DETAIL:  Key (usuario_id, videojuego_id)=(7, 1) already exists.
```

R11 · NOT NULL: usuario sin contraseña

```
INSERT INTO usuario (nombre, correo, rol)
VALUES ('Valeria Montoya', 'valeria.montoya@gmail.com', 'jugador');
ERROR:  null value in column "contrasena_hash" of relation "usuario" violates not-null constraint
DETAIL:  Failing row contains (18, Valeria Montoya, valeria.montoya@gmail.com, null, jugador, activo, 2026-11-03
19:45:36.728421-05, 0.00).
```

2.4 Procedimientos almacenados y triggers

En la Entrega 1 se marcaron cinco requerimientos con procedimiento almacenado o transacción explícita (RF-03, RF-06, RF-14, RF-15 y RF-17) y cinco reglas con trigger (RN-12 a RN-16). Se implementaron todos. El código completo está en el Anexo A y en `sql/03_procedimientos_triggers.sql`.

Caso de uso	Objeto	Tipo
RF-03 · Publicar un videojuego	`pr_publicar_videojuego`	PROCEDURE (una transacción)
RF-06 · Comprar un videojuego	`pr_comprar_videojuego`	PROCEDURE (una transacción)
RF-14 · Crear una promoción	`pr_crear_promocion`	PROCEDURE (una transacción)
RF-15 · Reporte de ventas del desarrollador	`fn_reporte_ventas_desarrollador`	FUNCTION … RETURNS TABLE
RF-17 · Reembolsar una compra	`pr_reembolsar_compra`	PROCEDURE (una transacción)

Triggers

Regla	Trigger	Momento	Qué hace
RN-12	`trg_rn12_compra_solo_publicados`	BEFORE INSERT ON compra	Rechaza la compra si el juego no está publicado.
RN-13	`trg_rn13_compra_no_duplicada`	BEFORE INSERT ON compra	Rechaza la compra si el juego ya está en la biblioteca del jugador.
RN-14	`trg_rn14_compra_a_biblioteca`	AFTER INSERT ON compra	Inserta el juego en `biblioteca` (efecto en otra tabla).
RN-15	`trg_rn15_resena_solo_propios`	BEFORE INSERT OR UPDATE ON resena	Rechaza la reseña si el juego no está en la biblioteca del autor.
RN-16	`trg_rn16_historial_precio`	AFTER UPDATE OF precio ON videojuego	Inserta precio anterior, nuevo y fecha en `historial_precio` (auditoría).

Ninguno de estos triggers puede reemplazarse por un CHECK: RN-12, RN-13 y RN-15 consultan otra tabla, y RN-14 y RN-16 escriben en otra tabla. Los procedimientos repiten algunas validaciones de los triggers para dar un mensaje más claro al usuario; los triggers garantizan la regla aunque alguien inserte directamente en la tabla sin usar el procedimiento.

Prueba de los triggers BEFORE (inserción directa, sin procedimiento)

RN-12 · comprar un juego retirado

```
INSERT INTO compra (usuario_id, videojuego_id, monto) VALUES (8, 11, 0.00);
ERROR:  No se puede comprar "Battle Arena Zero" (videojuego 11): su estado es retirado y solo se venden videojuegos publicados
CONTEXT:  PL/pgSQL function fn_rn12_compra_solo_publicados() line 12 at RAISE
```

RN-13 · comprar un juego que ya está en la biblioteca

```
INSERT INTO compra (usuario_id, videojuego_id, monto) VALUES (7, 1, 39.99);
ERROR:  El usuario 7 ya tiene el videojuego 1 en su biblioteca: no puede comprarlo de nuevo
CONTEXT:  PL/pgSQL function fn_rn13_compra_no_duplicada() line 7 at RAISE
```

RN-15 · reseñar un juego que no se posee

```
INSERT INTO resena (usuario_id, videojuego_id, calificacion, comentario)
VALUES (13, 1, 4, 'Me gustó mucho la historia.');
ERROR:  El usuario 13 no puede reseñar el videojuego 1 porque no está en su biblioteca
CONTEXT:  PL/pgSQL function fn_rn15_resena_solo_propios() line 7 at RAISE
```

Prueba de los triggers AFTER

RN-16 · el cambio de precio queda en el historial (y un UPDATE que no cambia el precio no genera fila, gracias a la condición `WHEN (OLD.precio IS DISTINCT FROM NEW.precio)`):

```
UPDATE videojuego SET estado = 'publicado' WHERE titulo = 'Leyendas del Amazonas';
UPDATE 1

UPDATE videojuego SET precio = 24.99 WHERE titulo = 'Leyendas del Amazonas';
UPDATE 1

SELECT h.id, h.videojuego_id, v.titulo, h.precio_anterior, h.precio_nuevo, h.fecha_cambio
  FROM historial_precio h JOIN videojuego v ON v.id = h.videojuego_id
 WHERE v.titulo = 'Leyendas del Amazonas';
 id | videojuego_id |        titulo         | precio_anterior | precio_nuevo |         fecha_cambio
----+---------------+-----------------------+-----------------+--------------+-------------------------------
  7 |            14 | Leyendas del Amazonas |           27.99 |        24.99 | 2026-11-03 19:45:36.739381-05
(1 row)

UPDATE videojuego SET precio = 24.99, descripcion = descripcion WHERE titulo = 'Leyendas del Amazonas';
UPDATE 1

SELECT count(*) AS cambios_de_precio
  FROM historial_precio h JOIN videojuego v ON v.id = h.videojuego_id
 WHERE v.titulo = 'Leyendas del Amazonas';
 cambios_de_precio
-------------------
                 1
(1 row)
```

RN-14 · un INSERT directo en compra agrega el juego a la biblioteca:

```
INSERT INTO compra (usuario_id, videojuego_id, monto) VALUES (15, 3, 14.99);
INSERT 0 1

SELECT usuario_id, videojuego_id, fecha_adquisicion FROM biblioteca WHERE usuario_id = 15 AND videojuego_id = 3;
 usuario_id | videojuego_id |       fecha_adquisicion
------------+---------------+-------------------------------
         15 |             3 | 2026-11-03 19:45:36.758192-05
(1 row)
```

Ficha RF-03 · Publicar un videojuego

Caso de uso	RF-03 · El desarrollador registra un videojuego con título, descripción, precio, fecha de lanzamiento, imágenes y uno o más géneros en una sola operación. El juego queda en estado pendiente (RN-07) hasta que un administrador lo publique (RF-13).
Objetos	`pr_publicar_videojuego` · restricciones `ck_videojuego_precio` (RN-05), `fk_videojuego_desarrollador` (RN-06) y `DEFAULT 'pendiente'` (RN-07) · `trg_rn16_historial_precio` registra los cambios de precio posteriores (RF-04).
Parámetros	• `p_desarrollador_id BIGINT`: desarrollador que publica. • `p_titulo VARCHAR`: título del juego. • `p_descripcion TEXT`: descripción. • `p_precio NUMERIC`: precio en dólares, mayor o igual a cero. • `p_fecha_lanzamiento DATE`: fecha de lanzamiento. • `p_generos BIGINT[]`: géneros del juego (al menos uno). • `p_imagenes TEXT[]`: URLs de las imágenes en el orden en que se muestran (opcional). • `INOUT p_videojuego_id BIGINT`: devuelve el id del juego creado (se pasa NULL).

Validaciones	• El desarrollador existe → "El desarrollador % no existe". • Su cuenta está activa → "La cuenta del desarrollador % está suspendida: no puede publicar videojuegos". • El título no está vacío → "El título del videojuego es obligatorio". • El desarrollador no tiene otro juego con ese título → "El desarrollador % ya tiene un videojuego llamado "%"". • Precio ≥ 0 → "El precio debe ser mayor o igual a cero: se recibió %". • Al menos un género → "El videojuego "%" debe tener al menos un género". • Cada género existe → "El género % no existe". • Ninguna imagen con URL vacía → "Las imágenes del videojuego "%" no pueden tener URL vacía".
Tablas afectadas	Lee `desarrollador`, `usuario`, `genero` y `videojuego`. Inserta en `videojuego`, `videojuego_genero` e `imagen_videojuego`.
Prueba de éxito	``` CALL pr_publicar_videojuego(5, 'Leyendas del Amazonas', 'Aventura de supervivencia río abajo por la selva amazónica, con fauna y comunidades reales como inspiración.', 27.99, '2027-03-15', ARRAY[6, 5]::BIGINT[], ARRAY['https://cdn.stanke.co/juegos/leyendas-del-amazonas/portada.jpg', 'https://cdn.stanke.co/juegos/leyendas-del-amazonas/rio.jpg'], NULL); NOTICE: Videojuego 14 "Leyendas del Amazonas" registrado en estado pendiente con 2 género(s) y 2 imagen(es) p_videojuego_id ----------------- 14 (1 row) SELECT v.id, v.titulo, v.precio, v.estado, d.nombre AS desarrollador FROM videojuego v JOIN desarrollador d ON d.id = v.desarrollador_id WHERE v.titulo = 'Leyendas del Amazonas'; id	titulo	precio	estado	desarrollador ----+-----------------------+--------+-----------+--------------- 14	Leyendas del Amazonas	27.99	pendiente	Nebula Soft (1 row) SELECT vg.videojuego_id, g.nombre AS genero FROM videojuego_genero vg JOIN genero g ON g.id = vg.genero_id WHERE vg.videojuego_id = (SELECT id FROM videojuego WHERE titulo = 'Leyendas del Amazonas'); videojuego_id	genero ---------------+---------- 14	Indie 14	Aventura (2 rows) SELECT videojuego_id, orden, url FROM imagen_videojuego WHERE videojuego_id = (SELECT id FROM videojuego WHERE titulo = 'Leyendas del Amazonas') ORDER BY orden; videojuego_id	orden	url ---------------+-------+--- 14	1	https://cdn.stanke.co/juegos/leyendas-del-amazonas/portada.jpg 14	2	https://cdn.stanke.co/juegos/leyendas-del-amazonas/rio.jpg (2 rows) ```
Prueba de error	``` CALL pr_publicar_videojuego(5, 'Ríos de Fuego', 'Plataformas en volcanes activos.', 15.99, '2027-05-01', ARRAY[6, 99]::BIGINT[], NULL, NULL); ERROR: El género 99 no existe CONTEXT: PL/pgSQL function pr_publicar_videojuego(bigint,character varying,text,numeric,date,bigint[],text[],bigint) line 43 at RAISE CALL pr_publicar_videojuego(99, 'Ríos de Fuego', 'Plataformas en volcanes activos.', 15.99, '2027-05-01', ARRAY[6]::BIGINT[], NULL, NULL); ERROR: El desarrollador 99 no existe CONTEXT: PL/pgSQL function pr_publicar_videojuego(bigint,character varying,text,numeric,date,bigint[],text[],bigint) line 13 at RAISE ```

	``` CALL pr_publicar_videojuego(1, 'Chronos Odyssey', 'Remasterización.', 59.99, '2027-01-01', ARRAY[2]::BIGINT[], NULL, NULL); ERROR: El desarrollador 1 ya tiene un videojuego llamado "Chronos Odyssey" CONTEXT: PL/pgSQL function pr_publicar_videojuego(bigint,character varying,text,numeric,date,bigint[],text[],bigint) line 27 at RAISE SELECT count(*) AS total_videojuegos FROM videojuego; total_videojuegos ------------------- 12 (1 row) ``` Ninguno de los intentos fallidos dejó filas: siguen existiendo 12 videojuegos.

Ficha RF-06 · Comprar un videojuego

Caso de uso	RF-06 · El jugador compra un videojuego. Se descuenta de su saldo el precio vigente (con la mejor promoción activa) y se registra la compra con fecha y monto; el juego pasa a su biblioteca y sale de su lista de deseos.
Objetos	`pr_comprar_videojuego` · triggers `trg_rn12_compra_solo_publicados` y `trg_rn13_compra_no_duplicada` (BEFORE), `trg_rn14_compra_a_biblioteca` (AFTER) · `ck_usuario_saldo` (RN-04).
Parámetros	• `p_usuario_id BIGINT`: jugador que compra. • `p_videojuego_id BIGINT`: juego que compra. • `INOUT p_compra_id BIGINT`: devuelve el id de la compra (se pasa NULL).
Validaciones	• El usuario existe → "El usuario % no existe". • Tiene rol jugador → "El usuario % tiene rol % y solo los jugadores pueden comprar videojuegos". • Su cuenta está activa → "La cuenta del usuario % está suspendida: no puede comprar videojuegos". • El juego existe → "El videojuego % no existe". • Está publicado (RN-12) → "El videojuego "%" no está a la venta: su estado es %". • No está ya en su biblioteca (RN-13) → "El usuario % ya tiene "%" en su biblioteca". • Saldo suficiente (RN-04) → "Saldo insuficiente: el usuario % tiene % y "%" cuesta %".
Tablas afectadas	Lee `usuario`, `videojuego`, `biblioteca`, `promocion` y `promocion_videojuego`. Modifica `usuario` (saldo), inserta en `compra` y, por el trigger RN-14, en `biblioteca`; borra de `lista_deseos`.
Prueba de éxito	Daniel (usuario 11, saldo 60.00) compra Sombras de Cartagena (34.99) con la promoción de 30 % creada en RF-14: paga 24.49, el juego queda en su biblioteca y sale de su lista de deseos. ``` SELECT id, nombre, saldo FROM usuario WHERE id = 11; id	nombre	saldo ----+---------------------+------- 11	Daniel Herrera Cruz	60.00 (1 row) SELECT usuario_id, videojuego_id FROM lista_deseos WHERE usuario_id = 11; usuario_id	videojuego_id ------------+--------------- 11	9 (1 row) CALL pr_comprar_videojuego(11, 9, NULL); NOTICE: Compra 31 registrada: "Sombras de Cartagena" por 24.49 (descuento 30.00 %); saldo restante 35.51 p_compra_id ------------- ```

	31 (1 row) SELECT id, nombre, saldo FROM usuario WHERE id = 11; id | nombre | saldo ----+----------+------- 11 | Daniel Herrera Cruz | 35.51 (1 row) SELECT id, usuario_id, videojuego_id, monto, estado FROM compra WHERE usuario_id = 11 AND videojuego_id = 9; id | usuario_id | videojuego_id | monto | estado ----+----------+---------------+-------+---------- 31 | 11 | 9 | 24.49 | completada (1 row) SELECT usuario_id, videojuego_id, fecha_adquisicion FROM biblioteca WHERE usuario_id = 11 AND videojuego_id = 9; usuario_id | videojuego_id | fecha_adquisicion ----------+---------------+---------- 11 | 9 | 2026-11-03 19:45:36.745658-05 (1 row) SELECT count(*) AS sigue_en_lista_deseos FROM lista_deseos WHERE usuario_id = 11 AND videojuego_id = 9; sigue_en_lista_deseos ---------- 0 (1 row)
Prueba de error	CALL pr_comprar_videojuego(13, 1, NULL); ERROR: Saldo insuficiente: el usuario 13 tiene 5.25 y "Chronos Odyssey" cuesta 39.99 CONTEXT: PL/pgSQL function pr_comprar_videojuego(bigint,bigint,bigint) line 59 at RAISE CALL pr_comprar_videojuego(9, 10, NULL); ERROR: El videojuego "Mechs of Titan" no está a la venta: su estado es pendiente CONTEXT: PL/pgSQL function pr_comprar_videojuego(bigint,bigint,bigint) line 41 at RAISE CALL pr_comprar_videojuego(7, 1, NULL); ERROR: El usuario 7 ya tiene "Chronos Odyssey" en su biblioteca CONTEXT: PL/pgSQL function pr_comprar_videojuego(bigint,bigint,bigint) line 46 at RAISE CALL pr_comprar_videojuego(14, 3, NULL); ERROR: La cuenta del usuario 14 está suspendida: no puede comprar videojuegos CONTEXT: PL/pgSQL function pr_comprar_videojuego(bigint,bigint,bigint) line 27 at RAISE SELECT id, nombre, saldo FROM usuario WHERE id = 13; id | nombre | saldo ----+----------+------- 13 | Mateo Salazar Quintero | 5.25 (1 row) El saldo de Mateo (13) sigue en 5.25: los intentos fallidos no cambiaron nada.

Ficha RF-14 · Crear una promoción

Caso de uso	RF-14 · El administrador crea una promoción con porcentaje de descuento, fecha de inicio, fecha de fin y los videojuegos a los que aplica, en una sola operación.
Objetos	`pr_crear_promocion` · restricciones `ck_promocion_descuento` y `ck_promocion_fechas` (RN-10).
Parámetros	• `p_nombre VARCHAR`: nombre comercial de la promoción. • `p_descuento NUMERIC`: porcentaje de descuento (1 a 90).

	• `p_fecha_inicio TIMESTAMPTZ`, `p_fecha_fin TIMESTAMPTZ`: periodo de vigencia. • `p_videojuegos BIGINT[]`: juegos incluidos (al menos uno). • `INOUT p_promocion_id BIGINT`: devuelve el id de la promoción (se pasa NULL).
Validaciones	• Nombre no vacío → "El nombre de la promoción es obligatorio". • Descuento entre 1 y 90 (RN-10) → "El descuento debe estar entre 1 % y 90 %: se recibió %". • Fechas presentes → "La promoción "%" necesita fecha de inicio y fecha de fin". • Fin posterior al inicio (RN-10) → "La fecha de fin (%) debe ser posterior a la fecha de inicio (%)". • La promoción no terminó ya → "No se puede crear la promoción "%": su fecha de fin (%) ya pasó". • Al menos un juego → "La promoción "%" debe aplicar al menos a un videojuego". • Cada juego existe → "El videojuego % no existe". • Ningún juego retirado → "El videojuego % "%" está retirado y no puede entrar en una promoción".
Tablas afectadas	Lee `videojuego`. Inserta en `promocion` y `promocion_videojuego`.
Prueba de éxito	`CALL pr_crear_promocion('Oferta Relámpago Stanke', 30, now(), now() + INTERVAL '7 days', ARRAY[8, 9]::BIGINT[], NULL);` `NOTICE: Promoción 8 "Oferta Relámpago Stanke" creada con 30 % de descuento para 2 videojuego(s)` ` p_promocion_id` `----------------` `8` `(1 row)` `SELECT p.id, p.nombre, p.descuento, v.titulo` `FROM promocion p` `JOIN promocion_videojuego pv ON pv.promocion_id = p.id` `JOIN videojuego v ON v.id = pv.videojuego_id` `WHERE p.nombre = 'Oferta Relámpago Stanke';` ` id	nombre	descuento	titulo` `----+-------------------------+-----------+----------------------` `8 | Oferta Relámpago Stanke | 30.00 | Turbo Drift Legends` `8 | Oferta Relámpago Stanke | 30.00 | Sombras de Cartagena` `(2 rows)`

Prueba de error	`CALL pr_crear_promocion('Mega descuento', 95, now(), now() + INTERVAL '3 days', ARRAY[1]::BIGINT[], NULL);`
	`ERROR: El descuento debe estar entre 1 % y 90 %: se recibió 95`
	`CONTEXT: PL/pgSQL function pr_crear_promocion(character varying,numeric,timestamp with time zone,timestamp with time zone,bigint[],bigint) line 10 at RAISE`
	
	`CALL pr_crear_promocion('Clásicos de Arena', 50, now(), now() + INTERVAL '3 days', ARRAY[2, 11]::BIGINT[], NULL);`
	`ERROR: El videojuego 11 "Battle Arena Zero" está retirado y no puede entrar en una promoción`
	`CONTEXT: PL/pgSQL function pr_crear_promocion(character varying,numeric,timestamp with time zone,timestamp with time zone,bigint[],bigint) line 43 at RAISE`
	
	`CALL pr_crear_promocion('Semana Retro', 20, '2026-12-10', '2026-12-01', ARRAY[6]::BIGINT[], NULL);`
	`ERROR: La fecha de fin (2026-12-01 00:00:00-05) debe ser posterior a la fecha de inicio (2026-12-10 00:00:00-05)`
	`CONTEXT: PL/pgSQL function pr_crear_promocion(character varying,numeric,timestamp with time zone,timestamp with time zone,bigint[],bigint) line 16 at RAISE`

Ficha RF-15 · Reporte de ventas del desarrollador

Caso de uso	RF-15 · Generar para el desarrollador el número de copias vendidas y los ingresos de cada uno de sus videojuegos en un periodo. Como devuelve filas, se implementa como función `RETURNS TABLE`. Incluye los juegos sin ventas y las copias reembolsadas, que no suman ingresos.

Objetos	`fn_reporte_ventas_desarrollador` (STABLE).
Parámetros	• `p_desarrollador_id BIGINT`: desarrollador del reporte. • `p_desde DATE`, `p_hasta DATE`: periodo, ambos días incluidos. • Devuelve: `videojuego_id`, `titulo`, `copias_vendidas`, `copias_reembolsadas`, `ingresos`.
Validaciones	• El desarrollador existe → "El desarrollador % no existe". • Fechas presentes → "Debe indicar la fecha inicial y la fecha final del periodo". • Periodo en orden → "Periodo inválido: la fecha final (%) es anterior a la inicial (%)".
Tablas afectadas	Lee `desarrollador`, `videojuego` y `compra`. No modifica ninguna tabla.

Prueba de éxito	`SELECT * FROM fn_reporte_ventas_desarrollador(1, '2026-01-01', '2026-12-31');`
	` videojuego_id |  titulo  | copias_vendidas | copias_reembolsadas | ingresos`
	`---+---+---+---+---`
	`  1 | Chronos Odyssey  | 5 | 0 | 191.95`
	`  6 | Pixel Dungeon Tactics | 3 | 0 | 26.47`
	`(2 rows)`
	
	`SELECT * FROM fn_reporte_ventas_desarrollador(5, '2025-01-01', '2026-12-31');`
	` videojuego_id |  titulo  | copias_vendidas | copias_reembolsadas | ingresos`
	`---+---+---+---+---`
	`  5 | Echoes of the Deep  | 4 | 1 | 72.46`
	`  14 | Leyendas del Amazonas | 0 | 0 | 0.00`
	`  9 | Sombras de Cartagena  | 0 | 1 | 0.00`
	`(3 rows)`
	
	En el segundo reporte se ve la compra de Sombras de Cartagena que se reembolsó en RF-17: cuenta como reembolsada y no suma ingresos.

Prueba de error	`SELECT * FROM fn_reporte_ventas_desarrollador(99, '2026-01-01', '2026-12-31');`
	`ERROR:  El desarrollador 99 no existe`
	`CONTEXT:  PL/pgSQL function fn_reporte_ventas_desarrollador(bigint,date,date) line 4 at RAISE`
	
	`SELECT * FROM fn_reporte_ventas_desarrollador(1, '2026-12-31', '2026-01-01');`
	`ERROR:  Periodo inválido: la fecha final (2026-01-01) es anterior a la inicial (2026-12-31)`
	`CONTEXT:  PL/pgSQL function fn_reporte_ventas_desarrollador(bigint,date,date) line 10 at RAISE`

Ficha RF-17 · Reembolsar una compra

Caso de uso	RF-17 · El jugador pide el reembolso de una compra de hace menos de 14 días: se devuelve el monto a su saldo, el juego sale de su biblioteca (y se borra su reseña, por RN-15) y la compra queda marcada como reembolsada.
Objetos	`pr_reembolsar_compra` · `ck_compra_estado`.
Parámetros	• `p_usuario_id BIGINT`: jugador que pide el reembolso. • `p_compra_id BIGINT`: compra a reembolsar.
Validaciones	• El usuario existe → "El usuario % no existe". • La compra existe → "La compra % no existe". • La compra es suya → "La compra % no pertenece al usuario %". • No fue reembolsada antes → "La compra % ya fue reembolsada". • Tiene menos de 14 días → "La compra % se hizo el % y ya pasaron más de 14 días: no se puede reembolsar".
Tablas afectadas	Lee y modifica `usuario` (saldo) y `compra` (estado). Borra de `biblioteca` y `resena`.

Prueba de éxito	Daniel pide el reembolso de la compra 31 hecha en la prueba de RF-06: recupera los 24.49 (saldo de nuevo en 60.00) y el juego sale de su biblioteca.
	`CALL pr_reembolsar_compra(11, 31);`
	`NOTICE: Compra 31 reembolsada: se devolvieron 24.49 al usuario 11`
	`CALL`
	
	`SELECT id, nombre, saldo FROM usuario WHERE id = 11;`
	` id | nombre | saldo`
	`----+--------------------+-------`
	` 11 | Daniel Herrera Cruz | 60.00`
	`(1 row)`
	
	`SELECT id, videojuego_id, monto, estado FROM compra WHERE id = 31;`
	` id | videojuego_id | monto | estado`
	`----+---------------+-------+-------------`
	` 31 | 9 | 24.49 | reembolsada`
	`(1 row)`
	
	`SELECT count(*) AS sigue_en_biblioteca FROM biblioteca WHERE usuario_id = 11 AND videojuego_id = 9;`
	` sigue_en_biblioteca`
	`---------------------`
	` 0`
	`(1 row)`

Prueba de error	`CALL pr_reembolsar_compra(11, 31);`
	`ERROR: La compra 31 ya fue reembolsada`
	`CONTEXT: PL/pgSQL function pr_reembolsar_compra(bigint,bigint) line 29 at RAISE`
	
	`CALL pr_reembolsar_compra(7, 1);`
	`ERROR: La compra 1 se hizo el 2025-12-02 y ya pasaron más de 14 días: no se puede reembolsar`
	`CONTEXT: PL/pgSQL function pr_reembolsar_compra(bigint,bigint) line 32 at RAISE`
	
	`CALL pr_reembolsar_compra(7, 3);`
	`ERROR: La compra 3 no pertenece al usuario 7`
	`CONTEXT: PL/pgSQL function pr_reembolsar_compra(bigint,bigint) line 26 at RAISE`

2.5 Diseño transaccional: RF-06 · Comprar un videojuego

Pregunta	Respuesta
Caso de uso elegido y por qué	RF-06, comprar un videojuego. Es la única operación en la que el jugador entrega dinero y la que alimenta los ingresos de los desarrolladores (RF-15) y las preguntas P1, P2, P7 y P9. Si sale mal, el jugador pierde saldo sin recibir el juego, recibe el juego sin pagarlo o paga dos veces el mismo juego; además los reportes de ventas quedan inflados o incompletos.
Operaciones de la transacción	Con autocommit, el CALL completo es una transacción. En orden: 1. `SELECT rol, estado, saldo FROM usuario WHERE id = p_usuario_id FOR UPDATE` (lee y bloquea al jugador). 2. `SELECT titulo, precio, estado FROM videojuego WHERE id = p_videojuego_id FOR SHARE`. 3. `SELECT 1 FROM biblioteca …` (RN-13) y `SELECT max(descuento) FROM promocion …` (precio vigente). 4. `UPDATE usuario SET saldo = saldo - monto` (el CHECK RN-04 se evalúa aquí). 5. `INSERT INTO compra …` → disparan `trg_rn12` y `trg_rn13` (BEFORE) y `trg_rn14` (AFTER), que hace `INSERT INTO biblioteca`. 6. `DELETE FROM lista_deseos …`. 7. COMMIT al terminar el CALL.

Pregunta	Respuesta
Si una operación falla	Cualquier RAISE EXCEPTION (de una validación, de un trigger o de una restricción) aborta la transacción completa y PostgreSQL deshace todo: el saldo vuelve a su valor, no queda la compra, el juego no entra a la biblioteca y la lista de deseos queda igual. La base queda exactamente como antes del CALL. Se comprobó en P12–P16 (el saldo de Mateo sigue en 5.25) y con la prueba de atomicidad A1–A3, que descuenta el saldo y luego falla en el INSERT por RN-13 (ver abajo).
Bloqueos	1) La fila del jugador en `usuario`, con `SELECT … FOR UPDATE`, antes de cualquier validación. Una segunda compra o reembolso del mismo jugador espera hasta el COMMIT de la primera. 2) La fila del juego en `videojuego`, con `FOR SHARE`: otros jugadores pueden comprar el mismo juego a la vez, pero nadie puede cambiar su precio o retirarlo mientras dura la compra. 3) Las filas nuevas de `compra` y `biblioteca` quedan bloqueadas por sus propios INSERT. El orden siempre es usuario → videojuego, y el reembolso usa usuario → compra; como todos empiezan por el usuario, no se forman ciclos de espera (deadlocks).
Nivel de aislamiento	READ COMMITTED, el predeterminado. Basta porque las decisiones que importan (saldo suficiente, juego no comprado antes) se toman después de bloquear la fila del jugador: quien espera el bloqueo, al continuar, lee en cada sentencia los datos ya confirmados por la otra transacción. Además, la llave primaria de `biblioteca` impide un duplicado aunque algo fallara. SERIALIZABLE obligaría a reintentar compras por errores de serialización sin aportar más protección para este caso.
Problema de concurrencia que evita	Compra duplicada (y doble cobro): el jugador hace clic dos veces o compra desde dos pestañas. Sin el FOR UPDATE, ambas sesiones verían que el juego no está en la biblioteca, ambas descontarían el saldo y ambas registrarían una compra. También evita la actualización perdida del saldo: dos compras simultáneas que leen el mismo saldo y lo validan con un valor viejo. Con el bloqueo, la segunda sesión espera, ve el juego ya comprado y falla con un mensaje claro (prueba con dos sesiones abajo).

Prueba de atomicidad (RNF-04)

Se descuenta el saldo de Carlos (7) dentro de una transacción y después se intenta registrar una compra que viola RN-13. Al fallar, el ROLLBACK deja el saldo igual:

```
SELECT id, nombre, saldo FROM usuario WHERE id = 7;
 id |       nombre       | saldo
----+--------------------+-------
  7 | Carlos Pérez Rojas | 85.50
(1 row)

BEGIN;
BEGIN

UPDATE usuario SET saldo = saldo - 39.99 WHERE id = 7;
UPDATE 1

INSERT INTO compra (usuario_id, videojuego_id, monto) VALUES (7, 1, 39.99);
ERROR:  El usuario 7 ya tiene el videojuego 1 en su biblioteca: no puede comprarlo de nuevo
CONTEXT:  PL/pgSQL function fn_rn13_compra_no_duplicada() line 7 at RAISE

ROLLBACK;
ROLLBACK

SELECT id, nombre, saldo FROM usuario WHERE id = 7;
 id |       nombre       | saldo
----+--------------------+-------
  7 | Carlos Pérez Rojas | 85.50
(1 row)
```

Prueba de concurrencia con dos sesiones

La sesión A abre una transacción, compra Sombras de Cartagena para Sebastián (15) y espera 5 segundos antes de confirmar. Un segundo después, la sesión B intenta la misma compra: queda bloqueada en el FOR UPDATE durante casi 4 segundos y, cuando A confirma, falla porque el juego ya está en la biblioteca. Resultado: una sola compra y un solo descuento de saldo (75.00 − 24.49 = 50.51). Salida real de las dos sesiones:

```
=== Antes
 id |        nombre         | saldo
----+-----------------------+-------
 15 | Sebastián Vargas León | 75.00
(1 row)



=== Sesión A
SELECT 'A inicia: ' || clock_timestamp()::time(3) AS t
           t
------------------------
 A inicia: 19:45:36.909
(1 row)

BEGIN
BEGIN
CALL pr_comprar_videojuego(15, 9, NULL)
NOTICE:  Compra 36 registrada: "Sombras de Cartagena" por 24.49 (descuento 30.00 %); saldo restante 50.51
 p_compra_id
-------------
          36
(1 row)

SELECT pg_sleep(5)
 pg_sleep
----------


(1 row)

SELECT 'A confirma: ' || clock_timestamp()::time(3) AS t
            t
--------------------------
 A confirma: 19:45:41.919
(1 row)

COMMIT
COMMIT

=== Sesión B
SELECT 'B inicia: ' || clock_timestamp()::time(3) AS t
           t
------------------------
 B inicia: 19:45:37.955
(1 row)

Timing is on.
CALL pr_comprar_videojuego(15, 9, NULL)
ERROR:  El usuario 15 ya tiene "Sombras de Cartagena" en su biblioteca
CONTEXT:  PL/pgSQL function pr_comprar_videojuego(bigint,bigint,bigint) line 46 at RAISE
Time: 3967.203 ms (00:03.967)
SELECT 'B termina: ' || clock_timestamp()::time(3) AS t
            t
-------------------------
 B termina: 19:45:41.922
(1 row)

Time: 0.149 ms

=== Después
 id |        nombre         | saldo
```

```
----+-----------------------+-------
 15 | Sebastián Vargas León | 50.51
(1 row)

 id | usuario_id | videojuego_id | monto |   estado
----+------------+---------------+-------+------------
 36 |         15 |             9 | 24.49 | completada
(1 row)
```

Anexo A. Código de 03_procedimientos_triggers.sql

```
-- ======================================================================
-- Stanke · 03_procedimientos_triggers.sql
-- Se ejecuta completo después de 01_create.sql y 02_datos.sql.
--
-- Triggers (reglas marcadas como trigger en la Entrega 1)
--   RN-12  BEFORE INSERT compra     · solo se compran juegos publicados
--   RN-13  BEFORE INSERT compra     · no se compra un juego que ya se posee
--   RN-14  AFTER  INSERT compra     · la compra agrega el juego a la biblioteca
--   RN-15  BEFORE INSERT/UPDATE resena · solo se reseñan juegos de la biblioteca
--   RN-16  AFTER  UPDATE precio     · todo cambio de precio queda en historial_precio
--
-- Casos de uso (requerimientos marcados con procedimiento o transacción)
--   RF-03  pr_publicar_videojuego
--   RF-06  pr_comprar_videojuego
--   RF-14  pr_crear_promocion
--   RF-15  fn_reporte_ventas_desarrollador  (consulta: RETURNS TABLE)
--   RF-17  pr_reembolsar_compra
-- ======================================================================



-- ======================================================================
-- TRIGGERS
-- ======================================================================


-- ----------------------------------------------------------------------
-- RN-12 · Solo se pueden comprar videojuegos en estado publicado (BEFORE)
-- ----------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_rn12_compra_solo_publicados()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_titulo VARCHAR(200);
    v_estado VARCHAR(20);
BEGIN
    SELECT titulo, estado INTO v_titulo, v_estado
      FROM videojuego
     WHERE id = NEW.videojuego_id;


    -- Si el videojuego no existe, la llave foránea lanzará su propio error.
    IF FOUND AND v_estado <> 'publicado' THEN
        RAISE EXCEPTION 'No se puede comprar "%" (videojuego %): su estado es % y solo se venden videojuegos publicados',
              v_titulo, NEW.videojuego_id, v_estado
              USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_rn12_compra_solo_publicados
BEFORE INSERT ON compra
FOR EACH ROW EXECUTE FUNCTION fn_rn12_compra_solo_publicados();


-- ----------------------------------------------------------------------
-- RN-13 · Un jugador no puede comprar un juego que ya está en su biblioteca (BEFORE)
-- ----------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_rn13_compra_no_duplicada()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF EXISTS (SELECT 1
                 FROM biblioteca
                WHERE usuario_id = NEW.usuario_id
                  AND videojuego_id = NEW.videojuego_id) THEN
        RAISE EXCEPTION 'El usuario % ya tiene el videojuego % en su biblioteca: no puede comprarlo de nuevo',
              NEW.usuario_id, NEW.videojuego_id
              USING ERRCODE = 'unique_violation';
    END IF;
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_rn13_compra_no_duplicada
BEFORE INSERT ON compra
FOR EACH ROW EXECUTE FUNCTION fn_rn13_compra_no_duplicada();


-- ----------------------------------------------------------------------
```

```
-- RN-14 · Al registrarse una compra, el juego se agrega a la biblioteca (AFTER)
-- ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_rn14_compra_a_biblioteca()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO biblioteca (usuario_id, videojuego_id, fecha_adquisicion)
    VALUES (NEW.usuario_id, NEW.videojuego_id, NEW.fecha);
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_rn14_compra_a_biblioteca
AFTER INSERT ON compra
FOR EACH ROW
WHEN (NEW.estado = 'completada')
EXECUTE FUNCTION fn_rn14_compra_a_biblioteca();


-- ------------------------------------------------------------------------
-- RN-15 · Un jugador solo puede reseñar un juego de su biblioteca (BEFORE)
-- ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_rn15_resena_solo_propios()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1
                     FROM biblioteca
                    WHERE usuario_id = NEW.usuario_id
                      AND videojuego_id = NEW.videojuego_id) THEN
        RAISE EXCEPTION 'El usuario % no puede reseñar el videojuego % porque no está en su biblioteca',
              NEW.usuario_id, NEW.videojuego_id
              USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_rn15_resena_solo_propios
BEFORE INSERT OR UPDATE OF usuario_id, videojuego_id ON resena
FOR EACH ROW EXECUTE FUNCTION fn_rn15_resena_solo_propios();

-- ------------------------------------------------------------------------
-- RN-16 · Todo cambio de precio queda registrado en historial_precio (AFTER)
-- ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_rn16_historial_precio()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO historial_precio (videojuego_id, precio_anterior, precio_nuevo)
    VALUES (NEW.id, OLD.precio, NEW.precio);
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_rn16_historial_precio
AFTER UPDATE OF precio ON videojuego
FOR EACH ROW
WHEN (OLD.precio IS DISTINCT FROM NEW.precio)
EXECUTE FUNCTION fn_rn16_historial_precio();



-- ========================================================================
-- PROCEDIMIENTOS Y FUNCIONES
-- ========================================================================


-- ------------------------------------------------------------------------
-- RF-03 · Publicar (registrar) un videojuego con sus géneros e imágenes
--         en una sola operación. El juego queda en estado 'pendiente'
--         (RN-07) hasta que un administrador lo apruebe (RF-13).
-- ------------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE pr_publicar_videojuego(
    p_desarrollador_id  BIGINT,
    p_titulo            VARCHAR,
    p_descripcion       TEXT,
    p_precio            NUMERIC,
    p_fecha_lanzamiento DATE,
    p_generos           BIGINT[],
    p_imagenes          TEXT[],
    INOUT p_videojuego_id BIGINT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
```

```
DECLARE
    v_estado_cuenta VARCHAR(20);
    v_genero        BIGINT;
BEGIN
    -- El desarrollador existe y su cuenta está activa
    SELECT u.estado INTO v_estado_cuenta
      FROM desarrollador d
      JOIN usuario u ON u.id = d.usuario_id
     WHERE d.id = p_desarrollador_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El desarrollador % no existe', p_desarrollador_id;
    END IF;
    IF v_estado_cuenta <> 'activo' THEN
        RAISE EXCEPTION 'La cuenta del desarrollador % está suspendida: no puede publicar videojuegos',
              p_desarrollador_id;
    END IF;

    -- Datos del videojuego
    IF p_titulo IS NULL OR btrim(p_titulo) = '' THEN
        RAISE EXCEPTION 'El título del videojuego es obligatorio';
    END IF;
    IF EXISTS (SELECT 1 FROM videojuego
                WHERE desarrollador_id = p_desarrollador_id
                  AND lower(titulo) = lower(btrim(p_titulo))) THEN
        RAISE EXCEPTION 'El desarrollador % ya tiene un videojuego llamado "%"',
              p_desarrollador_id, btrim(p_titulo);
    END IF;
    IF p_precio IS NULL OR p_precio < 0 THEN
        RAISE EXCEPTION 'El precio debe ser mayor o igual a cero: se recibió %', p_precio;
    END IF;

    -- Géneros: al menos uno y todos existentes
    IF p_generos IS NULL OR cardinality(p_generos) = 0 THEN
        RAISE EXCEPTION 'El videojuego "%" debe tener al menos un género', btrim(p_titulo);
    END IF;
    SELECT g.id_buscado INTO v_genero
      FROM unnest(p_generos) AS g(id_buscado)
     WHERE NOT EXISTS (SELECT 1 FROM genero WHERE id = g.id_buscado)
     LIMIT 1;
    IF FOUND THEN
        RAISE EXCEPTION 'El género % no existe', v_genero;
    END IF;

    -- Imágenes: opcionales, pero ninguna URL puede venir vacía
    IF p_imagenes IS NOT NULL
       AND EXISTS (SELECT 1 FROM unnest(p_imagenes) AS i(url)
                    WHERE i.url IS NULL OR btrim(i.url) = '') THEN
        RAISE EXCEPTION 'Las imágenes del videojuego "%" no pueden tener URL vacía', btrim(p_titulo);
    END IF;

    -- Inserciones (todas o ninguna)
    INSERT INTO videojuego (desarrollador_id, titulo, descripcion, precio, fecha_lanzamiento)
    VALUES (p_desarrollador_id, btrim(p_titulo), p_descripcion, p_precio, p_fecha_lanzamiento)
    RETURNING id INTO p_videojuego_id;            -- estado = 'pendiente' por DEFAULT (RN-07)

    INSERT INTO videojuego_genero (videojuego_id, genero_id)
    SELECT DISTINCT p_videojuego_id, g
      FROM unnest(p_generos) AS g;

    IF p_imagenes IS NOT NULL THEN
        INSERT INTO imagen_videojuego (videojuego_id, url, orden)
        SELECT p_videojuego_id, btrim(i.url), i.orden
          FROM unnest(p_imagenes) WITH ORDINALITY AS i(url, orden);
    END IF;

    RAISE NOTICE 'Videojuego % "%" registrado en estado pendiente con % género(s) y % imagen(es)',
          p_videojuego_id, btrim(p_titulo),
          (SELECT count(DISTINCT g) FROM unnest(p_generos) AS g),
          coalesce(cardinality(p_imagenes), 0);
END; $$;

-- ---------------------------------------------------------------------
-- RF-06 · Comprar un videojuego con el saldo de la cuenta, aplicando
--         la mejor promoción vigente. La compra dispara RN-12, RN-13 y
```

```
--         RN-14 (agrega el juego a la biblioteca).
-- ------------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE pr_comprar_videojuego(
    p_usuario_id    BIGINT,
    p_videojuego_id BIGINT,
    INOUT p_compra_id BIGINT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
DECLARE
    v_rol        VARCHAR(20);
    v_estado_usr VARCHAR(20);
    v_saldo      NUMERIC(12,2);
    v_titulo     VARCHAR(200);
    v_precio     NUMERIC(12,2);
    v_estado_vj  VARCHAR(20);
    v_descuento  NUMERIC(5,2);
    v_monto      NUMERIC(12,2);
BEGIN
    -- 1. Bloquea la fila del jugador: dos compras simultáneas del mismo
    --    jugador se ejecutan una después de la otra.
    SELECT rol, estado, saldo INTO v_rol, v_estado_usr, v_saldo
      FROM usuario
     WHERE id = p_usuario_id
       FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El usuario % no existe', p_usuario_id;
    END IF;
    IF v_rol <> 'jugador' THEN
        RAISE EXCEPTION 'El usuario % tiene rol % y solo los jugadores pueden comprar videojuegos',
              p_usuario_id, v_rol;
    END IF;
    IF v_estado_usr <> 'activo' THEN
        RAISE EXCEPTION 'La cuenta del usuario % está suspendida: no puede comprar videojuegos',
              p_usuario_id;
    END IF;

    -- 2. Lee el videojuego y evita que cambien su precio o estado mientras dura la compra
    SELECT titulo, precio, estado INTO v_titulo, v_precio, v_estado_vj
      FROM videojuego
     WHERE id = p_videojuego_id
       FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El videojuego % no existe', p_videojuego_id;
    END IF;
    IF v_estado_vj <> 'publicado' THEN                                  -- RN-12
        RAISE EXCEPTION 'El videojuego "%" no está a la venta: su estado es %', v_titulo, v_estado_vj;
    END IF;
    IF EXISTS (SELECT 1 FROM biblioteca
                WHERE usuario_id = p_usuario_id
                  AND videojuego_id = p_videojuego_id) THEN             -- RN-13
        RAISE EXCEPTION 'El usuario % ya tiene "%" en su biblioteca', p_usuario_id, v_titulo;
    END IF;

    -- 3. Precio vigente: se aplica el mayor descuento de las promociones activas
    SELECT max(p.descuento) INTO v_descuento
      FROM promocion p
      JOIN promocion_videojuego pv ON pv.promocion_id = p.id
     WHERE pv.videojuego_id = p_videojuego_id
       AND now() BETWEEN p.fecha_inicio AND p.fecha_fin;

    v_monto := round(v_precio * (1 - coalesce(v_descuento, 0) / 100), 2);

    IF v_saldo < v_monto THEN                                           -- RN-04
        RAISE EXCEPTION 'Saldo insuficiente: el usuario % tiene % y "%" cuesta %',
              p_usuario_id, v_saldo, v_titulo, v_monto;
    END IF;

    -- 4. Descuenta el saldo, registra la compra y limpia la lista de deseos
    UPDATE usuario
       SET saldo = saldo - v_monto
     WHERE id = p_usuario_id;

    INSERT INTO compra (usuario_id, videojuego_id, monto)
```

```
    VALUES (p_usuario_id, p_videojuego_id, v_monto)
    RETURNING id INTO p_compra_id;          -- trg_rn14 agrega el juego a biblioteca


    DELETE FROM lista_deseos
     WHERE usuario_id = p_usuario_id
       AND videojuego_id = p_videojuego_id;


    RAISE NOTICE 'Compra % registrada: "%" por % (descuento % %%); saldo restante %',
          p_compra_id, v_titulo, v_monto, coalesce(v_descuento, 0), v_saldo - v_monto;
END; $$;


-- ---------------------------------------------------------------------
-- RF-14 · Crear una promoción con sus videojuegos en una sola operación
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE pr_crear_promocion(
    p_nombre       VARCHAR,
    p_descuento    NUMERIC,
    p_fecha_inicio TIMESTAMPTZ,
    p_fecha_fin    TIMESTAMPTZ,
    p_videojuegos  BIGINT[],
    INOUT p_promocion_id BIGINT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
DECLARE
    v_juego  BIGINT;
    v_titulo VARCHAR(200);
BEGIN
    IF p_nombre IS NULL OR btrim(p_nombre) = '' THEN
        RAISE EXCEPTION 'El nombre de la promoción es obligatorio';
    END IF;
    IF p_descuento IS NULL OR p_descuento < 1 OR p_descuento > 90 THEN       -- RN-10
        RAISE EXCEPTION 'El descuento debe estar entre 1 %% y 90 %%: se recibió %', p_descuento;
    END IF;
    IF p_fecha_inicio IS NULL OR p_fecha_fin IS NULL THEN
        RAISE EXCEPTION 'La promoción "%" necesita fecha de inicio y fecha de fin', btrim(p_nombre);
    END IF;
    IF p_fecha_fin <= p_fecha_inicio THEN                                   -- RN-10
        RAISE EXCEPTION 'La fecha de fin (%) debe ser posterior a la fecha de inicio (%)',
              p_fecha_fin, p_fecha_inicio;
    END IF;
    IF p_fecha_fin <= now() THEN
        RAISE EXCEPTION 'No se puede crear la promoción "%": su fecha de fin (%) ya pasó',
              btrim(p_nombre), p_fecha_fin;
    END IF;
    IF p_videojuegos IS NULL OR cardinality(p_videojuegos) = 0 THEN
        RAISE EXCEPTION 'La promoción "%" debe aplicar al menos a un videojuego', btrim(p_nombre);
    END IF;


    -- Todos los videojuegos existen ...
    SELECT j.id_buscado INTO v_juego
      FROM unnest(p_videojuegos) AS j(id_buscado)
     WHERE NOT EXISTS (SELECT 1 FROM videojuego WHERE id = j.id_buscado)
     LIMIT 1;
    IF FOUND THEN
        RAISE EXCEPTION 'El videojuego % no existe', v_juego;
    END IF;


    -- ... y ninguno está retirado del catálogo
    SELECT v.id, v.titulo INTO v_juego, v_titulo
      FROM videojuego v
     WHERE v.id = ANY (p_videojuegos)
       AND v.estado = 'retirado'
     LIMIT 1;
    IF FOUND THEN
        RAISE EXCEPTION 'El videojuego % "%" está retirado y no puede entrar en una promoción',
              v_juego, v_titulo;
    END IF;


    INSERT INTO promocion (nombre, descuento, fecha_inicio, fecha_fin)
    VALUES (btrim(p_nombre), p_descuento, p_fecha_inicio, p_fecha_fin)
    RETURNING id INTO p_promocion_id;


    INSERT INTO promocion_videojuego (promocion_id, videojuego_id)
    SELECT DISTINCT p_promocion_id, j
      FROM unnest(p_videojuegos) AS j;
```

```
    RAISE NOTICE 'Promoción % "%" creada con % %% de descuento para % videojuego(s)',
          p_promocion_id, btrim(p_nombre), p_descuento,
          (SELECT count(DISTINCT j) FROM unnest(p_videojuegos) AS j);
END; $$;

-- ---------------------------------------------------------------------
-- RF-15 · Reporte de ventas de un desarrollador en un periodo.
--         Es una consulta que devuelve filas, por eso se implementa
--         como función RETURNS TABLE. Incluye juegos sin ventas.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_reporte_ventas_desarrollador(
    p_desarrollador_id BIGINT,
    p_desde            DATE,
    p_hasta            DATE
)
RETURNS TABLE (
    videojuego_id        BIGINT,
    titulo               VARCHAR,
    copias_vendidas      BIGINT,
    copias_reembolsadas  BIGINT,
    ingresos             NUMERIC
)
LANGUAGE plpgsql STABLE AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM desarrollador WHERE id = p_desarrollador_id) THEN
        RAISE EXCEPTION 'El desarrollador % no existe', p_desarrollador_id;
    END IF;
    IF p_desde IS NULL OR p_hasta IS NULL THEN
        RAISE EXCEPTION 'Debe indicar la fecha inicial y la fecha final del periodo';
    END IF;
    IF p_hasta < p_desde THEN
        RAISE EXCEPTION 'Periodo inválido: la fecha final (%) es anterior a la inicial (%)',
              p_hasta, p_desde;
    END IF;

    RETURN QUERY
    SELECT v.id,
           v.titulo,
           count(c.id) FILTER (WHERE c.estado = 'completada'),
           count(c.id) FILTER (WHERE c.estado = 'reembolsada'),
           coalesce(sum(c.monto) FILTER (WHERE c.estado = 'completada'), 0)::NUMERIC(12,2)
      FROM videojuego v
      LEFT JOIN compra c
             ON c.videojuego_id = v.id
            AND c.fecha >= p_desde
            AND c.fecha <  p_hasta + 1          -- incluye todo el día final
     WHERE v.desarrollador_id = p_desarrollador_id
     GROUP BY v.id, v.titulo
     ORDER BY 5 DESC, v.titulo;
END; $$;

-- ---------------------------------------------------------------------
-- RF-17 · Reembolsar una compra de menos de 14 días: devuelve el monto
--         al saldo, retira el juego de la biblioteca y marca la compra
--         como reembolsada. También elimina la reseña del jugador sobre
--         ese juego, para que siga cumpliéndose RN-15.
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE pr_reembolsar_compra(
    p_usuario_id BIGINT,
    p_compra_id  BIGINT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_dueno      BIGINT;
    v_juego      BIGINT;
    v_fecha      TIMESTAMPTZ;
    v_monto      NUMERIC(12,2);
    v_estado     VARCHAR(20);
BEGIN
    -- 1. Bloquea primero al usuario (mismo orden que la compra: usuario -> compra)
    PERFORM 1 FROM usuario WHERE id = p_usuario_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El usuario % no existe', p_usuario_id;
    END IF;

    -- 2. Bloquea la compra para que no se pueda reembolsar dos veces
```

```
    SELECT usuario_id, videojuego_id, fecha, monto, estado
      INTO v_dueno, v_juego, v_fecha, v_monto, v_estado
      FROM compra
     WHERE id = p_compra_id
       FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'La compra % no existe', p_compra_id;
    END IF;
    IF v_dueno <> p_usuario_id THEN
        RAISE EXCEPTION 'La compra % no pertenece al usuario %', p_compra_id, p_usuario_id;
    END IF;
    IF v_estado = 'reembolsada' THEN
        RAISE EXCEPTION 'La compra % ya fue reembolsada', p_compra_id;
    END IF;
    IF v_fecha < now() - INTERVAL '14 days' THEN
        RAISE EXCEPTION 'La compra % se hizo el % y ya pasaron más de 14 días: no se puede reembolsar',
              p_compra_id, to_char(v_fecha, 'YYYY-MM-DD');
    END IF;

    -- 3. Aplica el reembolso
    UPDATE usuario SET saldo = saldo + v_monto WHERE id = p_usuario_id;
    UPDATE compra  SET estado = 'reembolsada' WHERE id = p_compra_id;
    DELETE FROM resena
     WHERE usuario_id = p_usuario_id AND videojuego_id = v_juego;
    DELETE FROM biblioteca
     WHERE usuario_id = p_usuario_id AND videojuego_id = v_juego;

    RAISE NOTICE 'Compra % reembolsada: se devolvieron % al usuario %', p_compra_id, v_monto, p_usuario_id;
END; $$;
```

Anexo B. Ejecución de los scripts sobre una base vacía

Salida de `01`, `02` y `03` ejecutados con `ON_ERROR_STOP=1` (cualquier error detiene el script y cambia el código de salida). Los tres terminan con código 0:

```
PostgreSQL 18.1 on x86_64-windows, compiled by msvc-19.44.35219, 64-bit
=== 01_create.sql
DROP TABLE   (x14)
CREATE TABLE   (x11)
CREATE INDEX
CREATE TABLE   (x3)
CREATE INDEX   (x8)
=== 01_create.sql -> código de salida 0
=== 02_datos.sql
INSERT 0 15
INSERT 0 5
INSERT 0 8
INSERT 0 11
INSERT 0 18
INSERT 0 14
INSERT 0 6
INSERT 0 15
INSERT 0 6
INSERT 0 30
INSERT 0 28
INSERT 0 11
INSERT 0 20
INSERT 0 7
=== 02_datos.sql -> código de salida 0
=== 03_procedimientos_triggers.sql
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
CREATE TRIGGER
CREATE FUNCTION
```

```
CREATE TRIGGER
CREATE PROCEDURE   (x3)
CREATE FUNCTION
CREATE PROCEDURE
=== 03_procedimientos_triggers.sql -> código de salida 0
```